



A learned transceiver beats a matched classical receiver by **0.37 dB** on AWGN, gains **nothing** once the channel fades and CSI is estimated — and can be trained **with no channel model at all** for a measured **0.12–0.15 dB**.

1 Background

A conventional transceiver is a chain of separately designed blocks — each optimal alone, jointly only as good as the assumptions linking them. Deep learning replaces the chain with an **autoencoder** [1], or estimation and detection with a **learned receiver** [2]. Two objections stand in the way:

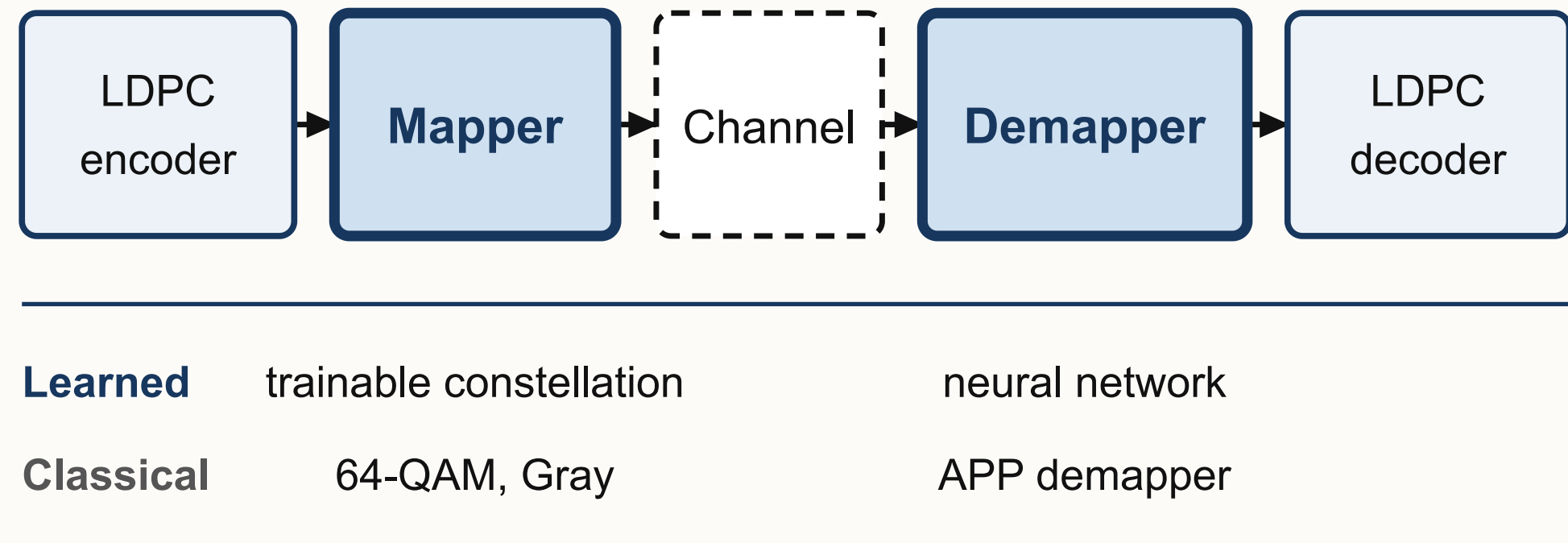
- Training needs a **differentiable channel model**. A deployed channel is a black box.
- Most reported gains are measured on **AWGN**, where the classical receiver is already optimal — so the comparison flatters the learned system.

This work answers both by measurement rather than by argument.

2 Objectives and Contributions

- Measure the gain of a learned transceiver against a **matched** classical baseline, on AWGN and on Rayleigh block fading with **pilot-estimated CSI**.
- Train the transmitter with **no channel model at all**, and price what that costs.
- Charge for everything: the transmitter never sees the channel, pilots cost rate $R(L-P)/L$, and estimation error enters as $N_0(1+1/P)/|\hat{h}|^2$ — omitting that last factor understates the noise by **1.76 dB** at $P = 2$.
- Diagnose failures **by intervention**: hold the weights fixed, change one thing, and see which change removes the symptom.

3 System Model



Channel: AWGN | Rayleigh block fading, $L = 25$, P pilot symbols, LS estimate

Fig. 1 — Only the two shaded blocks differ between the systems compared. The rate-1/2 5G LDPC code, the channel call and the random seeds are shared, so any difference in bit error rate is attributable to the mapper and demapper alone.

4 Where the AWGN Gain Comes From

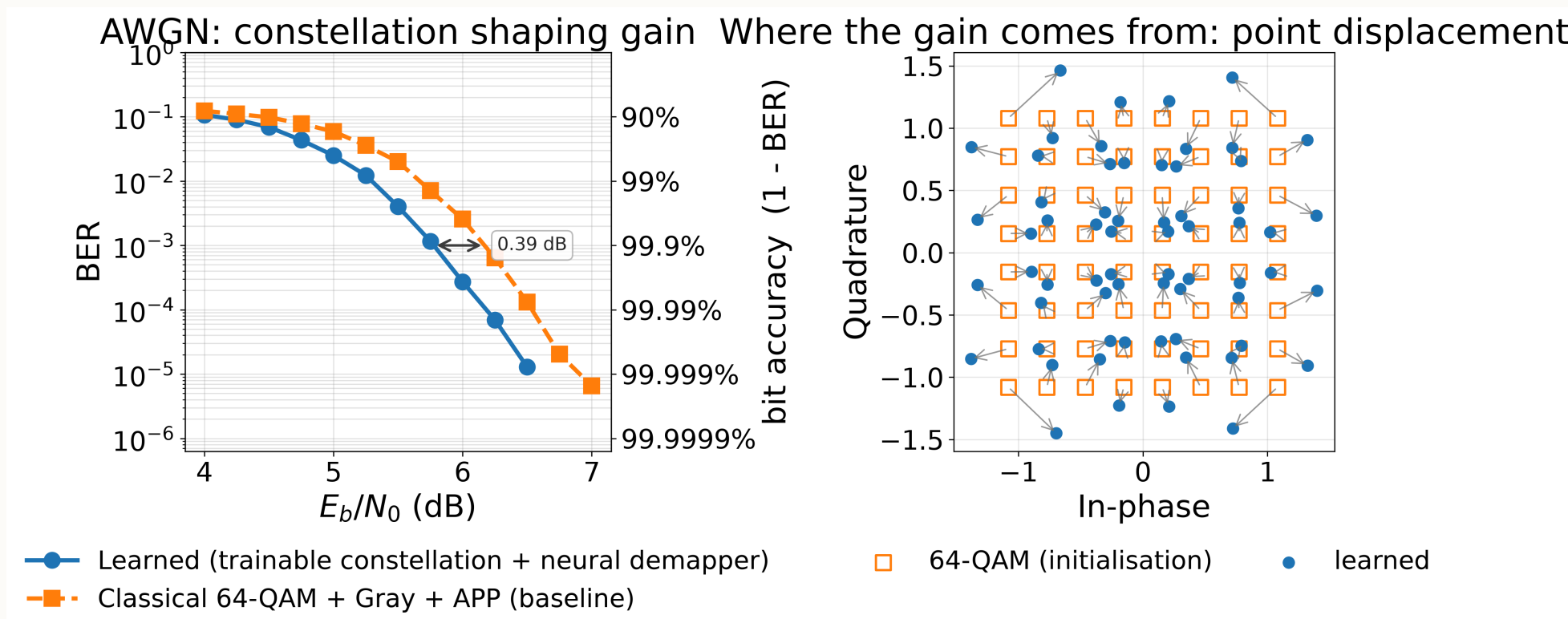


Fig. 2 — The learned system against 64-QAM with Gray labelling and APP demapping, on the same channel realisations.

- Under AWGN, APP demapping is **already optimal**, so the receiver cannot be the source of the gain.
- All of it comes from the transmitter: the learned constellation pulls off the rigid lattice, spending more power on the symbols that were most **confusable**.

5 Learning Without a Channel Model

Every result so far was trained by backpropagating **through** a differentiable channel; a deployed transmitter has none. Here the receiver trains normally — its gradient never crosses the channel — while the transmitter perturbs its own symbols and updates from **one scalar per codeword** returned by the receiver [4]. The channel is **sampled, never differentiated**.

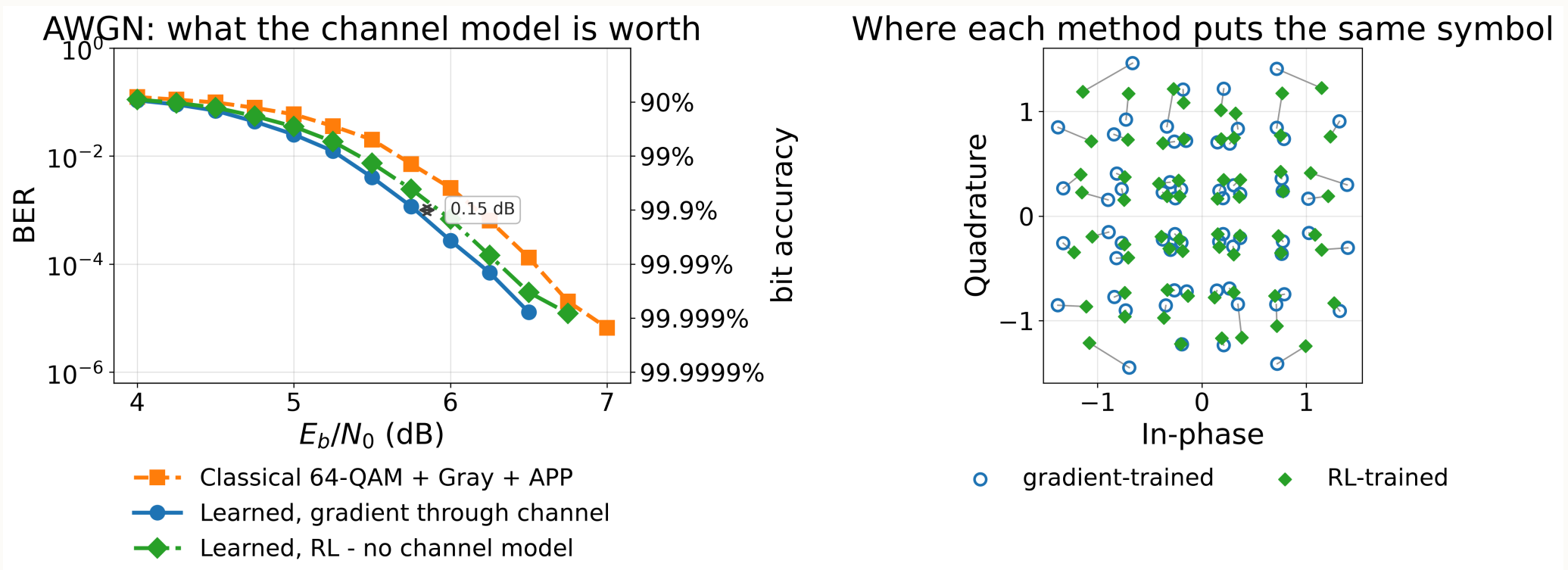


Fig. 3 — Reinforcement learning replaces the channel gradient at the transmitter. Lines in the right panel join the two positions of each symbol.

- **Price of dispensing with the channel model: +0.12 / +0.15 / +0.12 dB** at BER $1e-2$ / $1e-3$ / $1e-4$.
- It still beats classical 64-QAM by **0.24 dB**, with no channel model anywhere in training.
- Both methods put the same symbol in nearly the same place, so reinforcement learning recovers the gradient solution rather than a different one.

6 Both Channels, Both Receivers

Table 1 — E_b/N_0 required to reach a given bit error rate (dB, lower is better). 64,000 codewords per point; every entry rests on at least 30 block errors.

System	1e-2	1e-3	1e-4
AWGN, classical 64-QAM + APP	5.67	6.17	6.54
AWGN, learned (channel gradient)	5.30	5.78	6.18
AWGN, learned (RL, no channel model)	5.42	5.92	6.31
Rayleigh, classical 64-QAM + APP	11.26	13.30	14.79
Rayleigh, learned	11.25	13.19	14.94

- Fading costs **5.9–8.8 dB**. The choice of receiver moves things by under **0.4 dB** — an order of magnitude less.
- On Rayleigh with estimated CSI the learned system shows **no measurable gain** (+0.01 / +0.11 / –0.15 dB). The $1e-4$ entry **changed sign** between two runs of the same systems, so it is noise, not a difference.

7 Why AWGN Weights Fail on Fading

Reused on a fading channel without retraining, the AWGN weights give a BER curve that **rises** with SNR — non-physical, so something is wrong. Holding every weight fixed and changing one thing at a time isolates which:

- Clamping the network's **outputs**: under 4% change — the rise remains.
- Erasing deep-fade symbols: **worse** above 10 dB.
- Clamping the noise level **reported to the demapper** back into its training range: **the curve becomes physical**, with no weight changed.

So the fault is on the **input** side, and in the **quiet** tail rather than the deep-fade one: at 18 dB, **89%** of symbols carry an effective noise *lower* than anything seen in training, against 4.6% in deep fades. Retraining restores a monotonic curve — $1e-3$ at 13.2 dB, $1e-4$ at 14.9 dB.

8 Conclusion and Next Steps

Accuracy is **not** the case for an AI-native transceiver: the gain is real but small on AWGN and absent under fading. The case is that the design **needs no channel model** — and that now carries a measured price rather than an assumption.

- **RQ1** — **no channel model. Answered.** 0.12–0.15 dB against gradient training, still 0.24 dB ahead of the classical receiver.
- **RQ2** — **fast adaptation.** Remaining work. Meta-learning [3] targets exactly the distribution shift measured in §7 — motivation **measured here, not cited**.

Points resting on fewer than 30 block errors are dropped rather than drawn. Seeds fixed throughout; every number is measured in the accompanying notebook. Sionna [5] and PyTorch. **Scope:** flat block fading only — no multipath, Doppler or OFDM.

[1] T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer," arXiv:1702.00832, 2017.

[3] S. Smeka J. et al., "Accelerating Meta-Learning with the Enhanced Reptile Algorithm," SCSE, IEEE, 2025.

[5] NVIDIA Sionna, autoencoder tutorial notebook — the starting point this work extends.

[2] H. Ye, G. Y. Li and B.-H. F. Juang, "Power of Deep Learning for Channel Estimation and Signal Detection in OFDM," arXiv:1708.08514, 2017.

[4] N. C. Luong et al., "Applications of Deep Reinforcement Learning in Communications and Networking," IEEE Commun. Surveys Tuts., 21(4), 2019.